

INDEX

S. No.	Program Title	Page No.
1	Program to check the given number is prime or not	1
2	Program to check the given number is Armstrong	3
3	Program to find the factorial of the given number	5
4	Program to check the number is odd or even	7
5	Program using function	9
6	Program to find largest and smallest element in array	11
7	Sum of numbers in array	13
8	Even / Odd using array	15
9	Sum of 2D array	17
10	Program of Structure	19
11	Program to print table of the given number by user	21
12	Program array passed by reference	23
13	Inheritance	24
14	Multiple Inheritance	25
15	Multilevel inheritance	27
16	Hierarchical Inheritance	28
17	Constructor	30
18	Polymorphism	31
19	Function overloading	32
20	Access specifier	33
21	Class hierarchies	35
22	Class and object	37
23	Structure	38

S.no: 1

Program Title: Program to check the given number is prime or not

Source code

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int x;
    cout << "Please enter the no. here : ";
    cin >> x;

    if (x <= 1)
    {
        cout << x << " is not prime";
    }
    else
    {
        int limit = sqrt(x);
        bool isPrime = true;
        for (int i = 2; i <= limit; i++)
        {
            if (x % i == 0)
            {
                cout << x << " is not prime";
                isPrime = false;
                break;
            }
        }
        if (isPrime)
        {
            cout << "The number is prime";
        }
    }

    return 0;
}
```

TEST CASES:

Case 1;

Input = 2

```
● Please enter the no. here : 2
  The number is prime
```

Case 2;

Input = 10

```
Please enter the no. here : 10
10 is not prime
```

Case 3;

Input = 7

```
Please enter the no. here : 7
The number is prime
```

S.no: 2

Program Title: Program to check the given number is Armstrong

Source code

```
#include <iostream>
#include <cmath>
using namespace std;
bool isArmstrong(int);
int digitCount(int);
int main()
{
    for (int i = 0; i < 100000; i++)
    {
        if (isArmstrong(i))
        {
            cout << i << endl;
        }
    }
    return 0;
}
int digitCount(int x)
{
    if (x == 0)
        return 1;
    int dCount = 0;
    while (x > 0)
    {
        x /= 10;
        dCount++;
    }
    return dCount;
}
bool isArmstrong(int x)
{
    int value = x;
    int digits = digitCount(x);

    int sum = 0;

    while (x > 0)
    {
        sum += pow(x % 10, digits);
        x /= 10;
    }
    return value == sum;
}
```

Danishwer

0026MCA25

OUTPUT:

- PS C:\Users\danny\OneDrive\Desktop\MCA\CPP\AssignmentFile
programs\" ; if (\$?) { g++ armstrong.cpp -o armstrong } ;
0
1
2
3
4
5
6
7
8
9
370
371
407
1634
8208
9474
54748
92727
93084

S.no: 3

Program Title: Program to find the factorial of the given number

[Source code](#)

```
#include <iostream>
using namespace std;

int factorial(int n)
{
    return (n <= 1) ? 1 : n * factorial(n - 1);
}
int main()
{
    int n;
    cout << "Enter the number : ";
    cin >> n;
    cout << "Factorial of " << n << " is " << factorial(n);

    return 0;
}
```

TEST CASES:

Case 1;

Input = 5

```
1 // C++ program to find factorial of a number
2 #include <iostream>
3 using namespace std;
4 // Function to find factorial of a number
5 int factorial(int n)
6 {
7     if (n == 0)
8         return 1;
9     else
10        return n * factorial(n - 1);
11 }
12 // Driver code
13 int main()
14 {
15     int n = 5;
16     cout << "Factorial of " << n << " is " << factorial(n) << endl;
17     return 0;
18 }
```

- Enter the number : 5
Factorial of 5 is 120

Case 2;

Input = 6

```
1 // C++ program to find factorial of a number
2 #include <iostream>
3 using namespace std;
4 // Function to find factorial of a number
5 int factorial(int n)
6 {
7     if (n == 0)
8         return 1;
9     else
10        return n * factorial(n - 1);
11 }
12 // Driver code
13 int main()
14 {
15     int n = 6;
16     cout << "Factorial of " << n << " is " << factorial(n) << endl;
17     return 0;
18 }
```

- Enter the number : 6
Factorial of 6 is 720

Case 3;

Input = 0

- Enter the number : 0
Factorial of 0 is 1

S.no: 4

Program Title: Program to check the number is odd or even

Source code

```
#include <iostream>
using namespace std;
bool isEven(int x)
{
    return x % 2 == 0;
}

int main()
{
    int x;
    cout << "Enter the number here : ";
    cin >> x;
    cout << x << " is " << (isEven(x) ? "even" : "odd");
    return 0;
}
```


TEST CASES:

Case 1;

Input = 10

```
○ Enter the number here : 10
  10 is even
```

Case 2;

Input = 5

```
Enter the number here : 5
5 is odd
Enter the number here : 0
```

Case 3;

Input = 0

```
Enter the number here : 0
0 is even
```

—

S.no: 5

Program Title: Program using function

Source code

```
#include <iostream>
using namespace std;

int factorial(int n);

int main()
{
    int n;
    cout << "Enter the number : ";
    cin >> n;
    cout << "Factorial of " << n << " is " << factorial(n);

    return 0;
}

// Finding factorial using a function
int factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}
```

Danishwer

0026MCA25

TEST CASES:

Case 1;

Input = 6

```
● Enter the number : 6
  Factorial of 6 is 720
```

S.no: 6

Program Title: Program using function

Source code

```
#include <iostream>
#include "arrayprint.h"
using namespace std;

int min(int[], int);
int max(int[], int);
int main()
{
    // array input
    int arr[100], n;
    cout << "Enter number of elements: ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << "Min : " << min(arr, n) << endl;
    cout << "Min : " << max(arr, n);
    return 0;
}

int min(int ar[], int size)
{
    int mn = ar[0];
    for (int i = 0; i < size; i++)
    {
        int c = ar[i];
        mn = (mn > c) ? c : mn;
    }
    return mn;
}

int max(int ar[], int size)
{
    int mx = ar[0];
    for (int i = 0; i < size; i++)
    {
        mx = (mx < ar[i]) ? ar[i] : mx;
    }
    return mx;
}
```

TEST CASES:

Case 1;

Input = [1,2,3]

```
Enter number of elements: 3
```

```
1
```

```
2
```

```
3
```

```
Min : 1
```

```
Min : 3
```

S.no: 7

Program Title: Sum of numbers in array.

Source code

```
#include <iostream>
using namespace std;
int sum(int ar[], int size)
{
    int sm = 0;
    for (int i = 0; i < size; i++)
    {
        sm += ar[i];
    }
    return sm;
}
int main()
{
    // array input
    int arr[100], n;
    cout << "Enter number of elements: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << "Sum of the elements of the array : " << sum(arr, n);
    return 0;
}
```

TEST CASES:

Case 1;

Input = [1,2,3]

```
Enter number of elements: 3
```

```
1
```

```
2
```

```
3
```

```
Sum of the elements of the array : 6
```

S.no: 8

Program Title: Even / Odd using array.

Source code

```
#include <iostream>
using namespace std;

void even_fliter(int arr[], int size)
{
    cout << "Even numbers are : ";
    for (int i = 0; i < size; i++)
    {
        if (arr[i] % 2 == 0)
        {
            cout << arr[i] << ",";
        }
    }
    cout << "\n-----\n";
}

void odd_fliter(int arr[], int size)
{
    cout << "Odd numbers are : ";
    for (int i = 0; i < size; i++)
    {
        if (arr[i] % 2)
        {
            cout << arr[i] << ",";
        }
    }
    cout << "\n-----\n";
}

int main()
{
    int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    even_fliter(arr, 10);
    odd_fliter(arr, 10);
    return 0;
}
```


Output

array = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
1 g++ array-even-odd.cpp -o array-evi
● Even numbers are : 0,2,4,6,8,
-----
Odd numbers are : 1,3,5,7,9,
-----
- - - - -
```

S.no: 9

Program Title: Sum of 2d array.

Source code

```
#include <iostream>
using namespace std;
void print(int ar[][3])
{
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << ar[i][j] << "|";
        }
        cout << endl;
    }
}
void sum(int ar1[][3], int ar2[][3])
{
    int result[3][3];
    int range = 9;
    for (int i = 0; i < 9; i++)
    {
        result[i / 3][i % 3] = ar1[i / 3][i % 3] + ar2[i / 3][i % 3];
    }
    print(result);
}

int main()
{
    int ar1[3][3] = {{1, 2, 3}, {2, 3, 4}, {4, 5, 6}};
    int ar2[3][3] = {{3, 2, 1}, {4, 3, 2}, {6, 5, 4}};
    cout << "Array1" << endl;
    print(ar1);
    cout << "Array2" << endl;
    print(ar2);
    cout << "Sum " << endl;
    sum(ar1, ar2);
    return 0;
}
```

Output

● Array1

○ 1|2|3|

2|3|4|

4|5|6|

Array2

3|2|1|

4|3|2|

6|5|4|

Sum

4|4|4|

6|6|6|

10|10|10|

S.no: 10

Program Title: Program of Structure

Source code

```
#include <iostream>
using namespace std;

struct Person
{
    string name;
    int age;
    void show_details()
    {
        cout << name << "'s age is " << age;
    }
};

int main()
{
    struct Person p;
    p.name = "John";
    p.age = 10;
    p.show_details();
    return 0;
}
```

Output

```
g++ structure.cpp
● John's age is 10
```

S.no: 11

Program Title: Program to print table of the given number by user.

Source code

```
#include <iostream>
using namespace std;

void table_print(int number)
{
    for (int i = 1; i <= 10; i++)
    {
        cout << number << " x " << i << " = " << number * i << endl;
    }
}

int main()
{
    int i;
    cout << "Enter the number here: ";
    cin >> i;
    table_print(i);
    return 0;
}
```

Test Cases

Case 1;

input =1

● Enter the number here: 1

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
```

Case 2;

input =16

Enter the number here: 16

```
16 x 1 = 16
16 x 2 = 32
16 x 3 = 48
16 x 4 = 64
16 x 5 = 80
16 x 6 = 96
16 x 7 = 112
16 x 8 = 128
16 x 9 = 144
16 x 10 = 160
```

S.no: 12

Program Title: Program array passed by reference.

Source code

```
#include <iostream>
using namespace std;

int double_elements(int array[], int size)
{
    for (int i = 0; i < size; i++)
    {
        array[i] = array[i] * 2;
    }
}

int main()
{
    int arr[3] = {1, 2, 3};
    cout << "Before the Call\n";
    for (int e : arr)
    {
        cout << e << ",";
    }
    double_elements(arr, 3);

    cout << "\nAfter the Call\n";
    // the array is modified here too
    for (int e : arr)
    {
        cout << e << ",";
    }
    return 0;
}
```

OUTPUT

```
Before the Call
1,2,3,
After the Call
2,4,6,
```

S.no: 13

Program Title: Single Level Inheritance

```
// Single Level Inheritance
#include <iostream>
using namespace std;

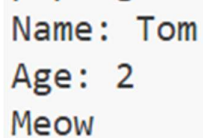
class Animal
{
public:
    string name;
    int age;
    void details()
    {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

class Cat : public Animal
{
public:
    void meow()
    {
        cout << "Meow" << endl;
    }
};

int main()
{
    Cat c1;
    c1.name = "Tom";
    c1.age = 2;
    c1.details();
    c1.meow();

    return 0;
}
```

OUTPUT

A screenshot of a terminal window showing the output of the C++ program. The output consists of three lines: "Name: Tom", "Age: 2", and "Meow".

```
Name: Tom
Age: 2
Meow
```

Danishwer

0026MCA25

S.no: 14

Program Title: Multiple Inheritance

```
// Multiple Inheritance
#include <iostream>
using namespace std;

class Human
{
public:
    int age;
    Human(int age) : age(age) {}
    virtual void introduce()
    {
        cout << "Age is " << age << endl;
    }
};

class Employee
{
public:
    string name;
    float salary;
    string company;

    Employee(string name, float salary, string company)
        : name(name), salary(salary), company(company) {}
    void introduce()
    {
        cout << "Name is " << name << endl;
        cout << "Salary is " << salary << endl;
        cout << "Company is " << company << endl;
    }
};

class Engineer : public Human, public Employee
{
public:
    int experience;
    Engineer(string name, int age, float salary, string company, int
experience) : Human(age), Employee(name, salary, company)
    {
        this->experience = experience;
    }
    void introduce() override
    {
```

Danishwer

0026MCA25

```
        cout << "Name is " << name << endl;
        cout << "Age is " << age << endl;
        cout << "Salary is " << salary << endl;
        cout << "Company is " << company << endl;
        cout << "Experience is " << experience << endl;
    }
};

int main()
{
    Engineer s1("Danny", 20, 100000, "Google", 5);
    s1.introduce();
    return 0;
}
```

OUTPUT

```
Name is Danny
Age is 20
Salary is 100000
Company is Google
Experience is 5
```

S.no: 15

Program Title: Multilevel inheritance

```
// Multilevel inheritance
#include <iostream>
using namespace std;
class Animal
{
public:
    void eat()
    {
        cout << "Eating" << endl;
    }
};
class LandAnimal : public Animal
{
public:
    void walk()
    {
        cout << "Walking" << endl;
    }
};
class Dog : public LandAnimal
{
public:
    void bark()
    {
        cout << "Barking" << endl;
    }
};
int main()
{
    Dog d;
    d.eat();
    d.walk();
    d.bark();
    return 0;
}
```

OUTPUT

```
Eating
Walking
Barking
```

Danishwer

0026MCA25

S.no: 16

Program Title: Hierarchical Inheritance

```
// Hierarchical Inheritance
#include <iostream>
using namespace std;
class Shape
{
public:
    int width, height;
    Shape(int w, int h)
    {
        width = w;
        height = h;
    }
};
class Rectangle : public Shape
{
public:
    Rectangle(int w, int h) : Shape(w, h) {}
    int area()
    {
        return width * height;
    }
};
class Triangle : public Shape
{
public:
    Triangle(int w, int h) : Shape(w, h) {}
    int area()
    {
        return (width * height) / 2;
    }
};

int main()
{
    Rectangle r(10, 20);
    Triangle t(10, 20);
    cout << r.area() << endl;
    cout << t.area() << endl;
    return 0;
}
```

Danishwer

0026MCA25

OUTPUT

☒ 200
☐ 100

S.no: 17

Program Title: Constructor

```
// Constructor
#include <iostream>
using namespace std;

class Person
{
    string name;
    int age;

public:
    Person(string n, int a)
    {
        cout << "Constructor called" << endl;
        name = n;
        age = a;
    }
    void introduce()
    {
        cout << "Name is " << name << endl;
        cout << "Age is " << age << endl;
    }
};

int main()
{
    Person p1("John", 20); // constructor called
    p1.introduce();
    return 0;
}
```

OUTPUT

```
Constructor called
Name is John
Age is 20
```

Danishwer

0026MCA25

S.no: 18

Program Title: Polymorphism

```
// Polymorphism
#include <iostream>
using namespace std;

class Adder
{
public:
    static int add(int a, int b)
    {
        return a + b;
    }
    static int add(int a, int b, int c)
    {
        return a + b + c;
    }
    static double add(double a, double b)
    {
        return a + b;
    }
};

int main()
{
    cout << Adder::add(1, 2) << endl;
    cout << Adder::add(1, 2, 3) << endl;
    cout << Adder::add(1.0, 2.0) << endl;
    return 0;
}
```

OUTPUT

```
3
6
3
```

Danishwer

0026MCA25

S.no: 19

Program Title: Function overloading

```
// Function overloading
#include <iostream>
using namespace std;

int add(int a, int b)
{
    return a + b;
}
int add(int a, int b, int c)
{
    return a + b + c;
}
double add(double a, double b)
{
    return a + b;
}
int main()
{
    cout << add(10, 20) << endl;
    cout << add(10, 20, 30) << endl;
    cout << add(10.0, 20.0) << endl;
    return 0;
}
```

OUTPUT

```
30
60
30
```


S.no: 20

Program Title: Access specifier

```
// Access specifier

#include <iostream>
using namespace std;

class A
{
private:
    int pri;

protected:
    int pro;

public:
    int pub;
    void show()
    {
        cout << "public : " << pub << endl;
        cout << "protected : " << pro << endl;
        cout << "private : " << pri << endl;
    }
};

class B : public A
{
public:
    void modify()
    {
        pub = 1;
        pro = 3;
        // pri = 2; //not accessible
    }
};

int main()
{
    A obj;
    obj.pub = 1;
    // obj.pri = 2; //not accessible
    // obj.pro = 3; //not accessible
    obj.show();
}
```

Danishwer

0026MCA25

```
B obj1;  
obj1.modify();  
obj1.show();  
return 0;  
}
```

OUTPUT

```
public : 1  
protected : 0  
private : 17569456  
public : 1  
protected : 3  
private : 6422356
```

S.no: 21

Program Title: Class hierarchies

```
// Class hierarchies:
// Hybrid inheritance and Dimond problem
#include <iostream>
using namespace std;
class Animal
{
public:
    void eat()
    {
        cout << "Eating" << endl;
    }
};

// using virtual inheritance to avoid diamond problem
class LandAnimal : virtual public Animal
{
public:
    void walk()
    {
        cout << "Walking" << endl;
    }
};
class Mammal : virtual public Animal
{
public:
    void feed()
    {
        cout << "Feeding" << endl;
    }
};

class Cat : public Mammal, public LandAnimal
{
public:
    void meow()
    {
        cout << "Meow" << endl;
    }
};
```

Danishwer

0026MCA25

```
int main()
{
    Cat c1;
    c1.eat();
    c1.walk();
    c1.feed();
    c1.meow();
    return 0;
}
```

OUTPUT

```
Eating
Walking
Feeding
Meow
```

S.no: 22

Program Title: Class and object

```
// Class and object
#include <iostream>
using namespace std;
class Human
{
public:
    int age;
    string name;
    void introduce()
    {
        cout << "Age is " << age << endl;
        cout << "Name is " << name << endl;
    }
};
int main()
{
    Human h1;
    h1.age = 20;
    h1.name = "John";
    h1.introduce();
    return 0;
}
```

OUTPUT

```
Age is 20
Name is John
```

S.no: 23

Program Title: C Style struct

```
// Structure
#include <iostream>
using namespace std;
struct Person
{
    char name[20];
    int age;
};

void show_details(Person p)
{
    cout << p.name << "'s age is " << p.age << endl;
}

int main()
{
    Person people[3];
    for (int i = 0; i < 3; i++)
    {
        Person p;
        cout << "Enter name: ";
        cin >> p.name;
        cout << "Enter age: ";
        cin >> p.age;
        people[i] = p;
    }

    for (Person p : people)
    {
        show_details(p);
    }

    return 0;
}
```

Danishwer

0026MCA25

OUTPUT

```
Enter name: Danny
Enter age: 22
Enter name: Raju
Enter age: 18
Enter name: Ron
Enter age: 12
Danny's age is 22
Raju's age is 18
Ron's age is 12
```